Lab 5 Fillable Worksheet

Debugging & Network Traffic Analysis

Complete this worksheet while performing your malware analysis. Fill in each section as you work. When finished, use **Print / Save as PDF** to submit your completed handout.

> **Safety reminder:** Analyze malware only inside an isolated virtual machine or a controlled sandbox such as **ANY.RUN**. Never run malware on your host system.

Student Information

Student name

Tyler Vander Mooren

Date

04 / 06 / 2026

Course / section

CYB310-002

Instructor

Troy Adams

Sample Identification

Malware sample name

Agent Tesla

Source repository

MalwareBazaar

File name

gnxLZ.exe

File size

323'584 bytes

SHA-256 hash

6f25b64efa6c3595eccd87c8c3a1f5265950b3f64bdcde882338ad9d84712f02

Known or suspected malware family

Agent Tesla / Stealer

Environment Preparation

Preparation item	Done	Notes

Preparation item	Done	Notes
Created a VM snapshot before execution	Yes	Windows 10 VM using VirtualBox.
Configured isolated networking	Yes	VM configured using NAT.
Disabled shared folders and unsafe host integration	Yes	
Started monitoring tools before execution	Yes	X32dbg for debugging, Wireshark for network capture and ProcMon.

Debugger Observations

Entry point address

Not cleanly identifiable

Debugger used

x32dbg

Suspicious API	Why it matters	Observed behavior / parameters
NtUserTranslateMessage	GUI/message loop API seen early in execution; not malicious itself but confirms program initialization	Observed during early execution in `win32u.dll`, indicating transition through Windows message handling.
NtUserPostMessage	Used for inter-process or GUI messaging; can be abused for stealth communication.	Seen during runtime transition, no clear malicious parameters yet.
NtUserQueryWindow	Can be used to enumerate windows or interact with UI elements.	Indicates interaction with system environment.
NtUserRedrawWindow	GUI-related, often noise but confirms execution flow	Observed in system-level execution chain.
NtUserWindowFromPoint	Can be used for UI targeting or interaction. Windows Error	Seen during execution, part of system call chain.
WerRegisterMemoryBlock	Reporting API; sometimes abused to register memory regions.	Seen in `kernelbase.dll`, indicates memory interaction or runtime setup.

Breakpoints that triggered

No manual breakpoints were triggered on user defined malware code, but the execution breakpoints consistently landed inside ntdll.dll, win32u.dll, kernelbase.dll. This can indicate that the malware did not expose direct native execution flow.

Processes spawned or injected into

No process spawning or injection was observed during debugging, indicating likely delayed or hidden.

Memory allocation, unpacking, or code injection findings

There was no explicit unpacking routine observed in x32dbg, also there was no clear memory injection behavior identified during stepping. However, the execution path does suggest that the malware may rely on managed execution and obfuscation rather than unpacking techniques.

System Activity (ProcMon or Equivalent)

Artifact type	Observed evidence
Files created or modified	No clearly malicious file creation or modification was confirmed by the malware process in the provided log. The sample mainly opened existing files and attempted to locate related .config and .INI files.
Registry keys modified	No confirmed registry value creation or modification was observed. The log showed extensive registry reads and open, but no clear RegSetValue activity.
Persistence mechanisms	None observed.
Processes created	No child process creation by the malware was observed.

Network Activity (Wireshark, FakeNet-NG, or ANY.RUN)

Observation	Details
DNS queries	20 DNS requests were observed from ANY.RUN, but these processes were mostly associated with background Window processes rather than the malware itself.
Domains contacted	Visible domains included settings-win.data.microsoft.com, login.live.com, ocsp.digicert.com, oneocsp.microsoft.com, crl.microsoft.com, www.microsoft.com, slscr.update.microsoft.com, and fe3cr.delivery.mp.microsoft.com, all shown as whitelisted in the ANY.RUN report.
IP addresses contacted	Visible IPs included 20.190.160.65, 4.231.128.59, 51.124.78.146, 20.73.194.208, 23.11.41.157, 204.79.197.203, but these were largely Windows/background network traffic.
Protocols observed	HTTP and HTTPS traffic was present in the sandbox environment along with DNS activity.
Possible command-and-control behavior	Network activity occurred in the environment, but the visible report pages do not clearly prove malware specific command-and-control communication. Most listed network traffic appears to be normal Windows services and are marked whitelisted.

Indicators of Compromise (IOCs)

Indicator type	Value
File hash	6f25b64efa6c3595eccd87c8c3a1f5265950b3f64bdcde882338ad9d84712f02
Domain	None observed
IP address	None observed

Indicator type	Value
Registry key	HKLM\SOFTWARE\Microsoft\Cryptography\MachineGuid HKLM\System\CurrentControlSet\Control\ComputerName\ActiveComputerName\ComputerName HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\User Shell Folders\AppData
Dropped file	None observed
Mutex, service, or task name	None observed

Behavioral Summary and Reflection

Briefly summarize what the malware does when executed

During execution the malware performed host reconnaissance by reading the machine GUID and computer name, which is consistent with victim identification and staging for credential theft or surveillance. Although network activity was present in the sandbox, the visible report pages did not clearly attribute command-and-control traffic directly to the malware process.

What evidence most strongly indicated malicious behavior?

The strongest evidence was from ANY.RUN where it revealed YARA detection of Agent Tesla, classification as a stealer, the malware's reading of host identifying registry information such as the machine GUID and computer name. In addition there was the presence of .NET Reactor protection, which is commonly used to hinder analysis of malicious .NET samples. Which could indicate why ProcMon and x32dbg didn't confirm any malicious behavior just suspicious.

What indicators could defenders use to detect this malware?

Defenders could use the executable SHA-256 hash, the dropped executable path, Agent Tesla family identification, detection of suspicious .NET-protected executable and monitoring for processes that read the machine GUID and computer name shortly after execution. Using those host based behaviors and combing it with known Agent Tesla detections would be strong indicator points to use.

Did ANY.RUN or another sandbox reveal behavior you did not initially see manually?

ANY.RUN strengthened the analysis significantly by explicitly identifying the sample as Agent Tesla and a stealer, confirming that it reads the machine GUID and computer name and detecting .NET Reactor protection. Where in my VM environment, x32dbg wasn't able to clearly identify and ProcMon didn't show any confirmed creation of artifacts.

After completing the form, choose **Print / Save as PDF** and select **Save as PDF** in your browser's print dialog.